

2D Arrays

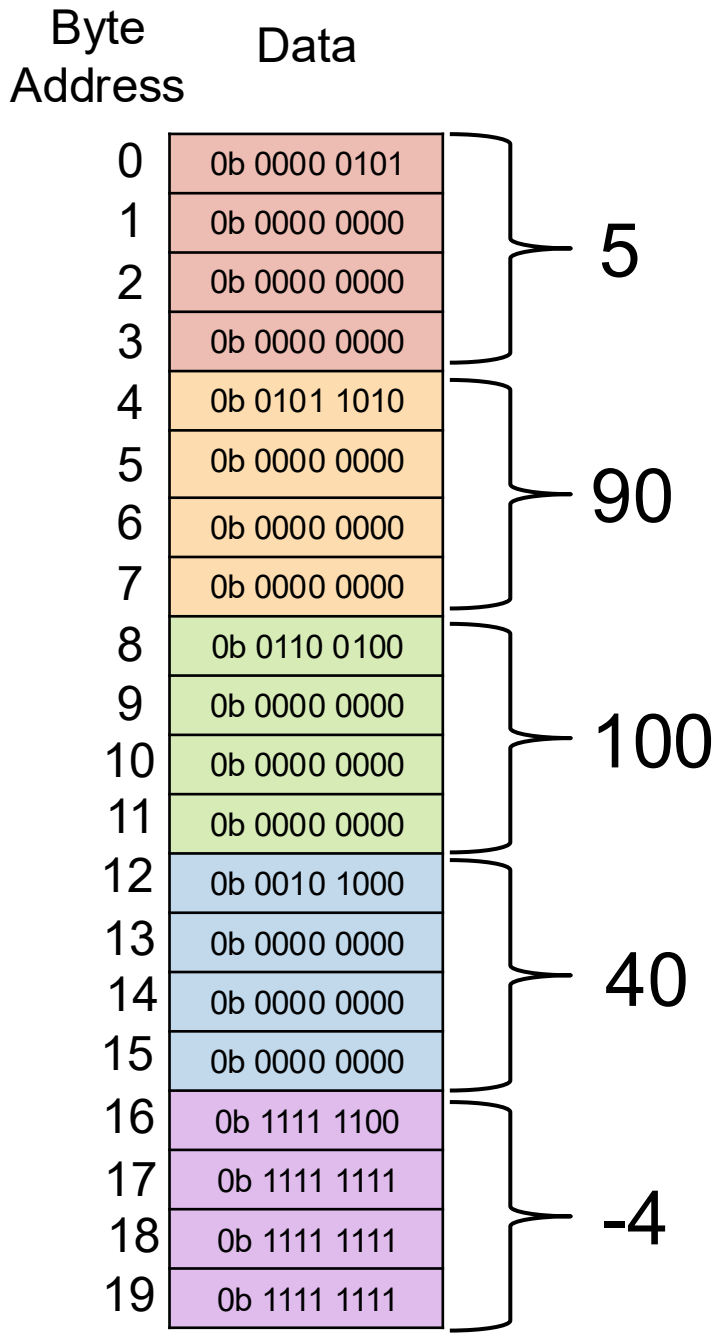
COMP211 Connor McMahon

Review: Arrays

- Arrays are stored **contiguously** in memory
  - All the elements of an array are placed in consecutive memory locations.

0	1	2	3	4
5	90	100	40	-4

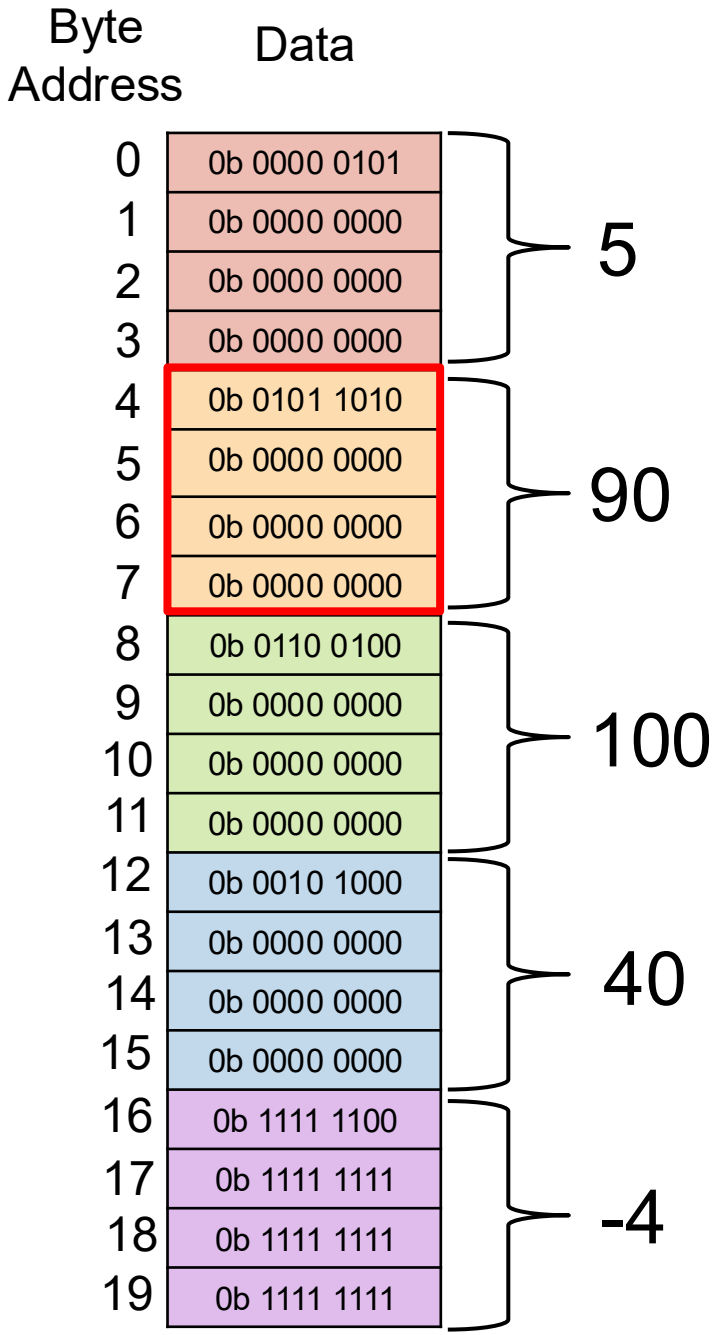
Example: Array of `int32_t`



Arrays

- A **word** is the standard unit of data that a computer's processor can store, process, or transfer as a single unit.
  - On a 32-bit computer, a word = 32 bits
  - On a 64-bit computer, a word = 64 bits

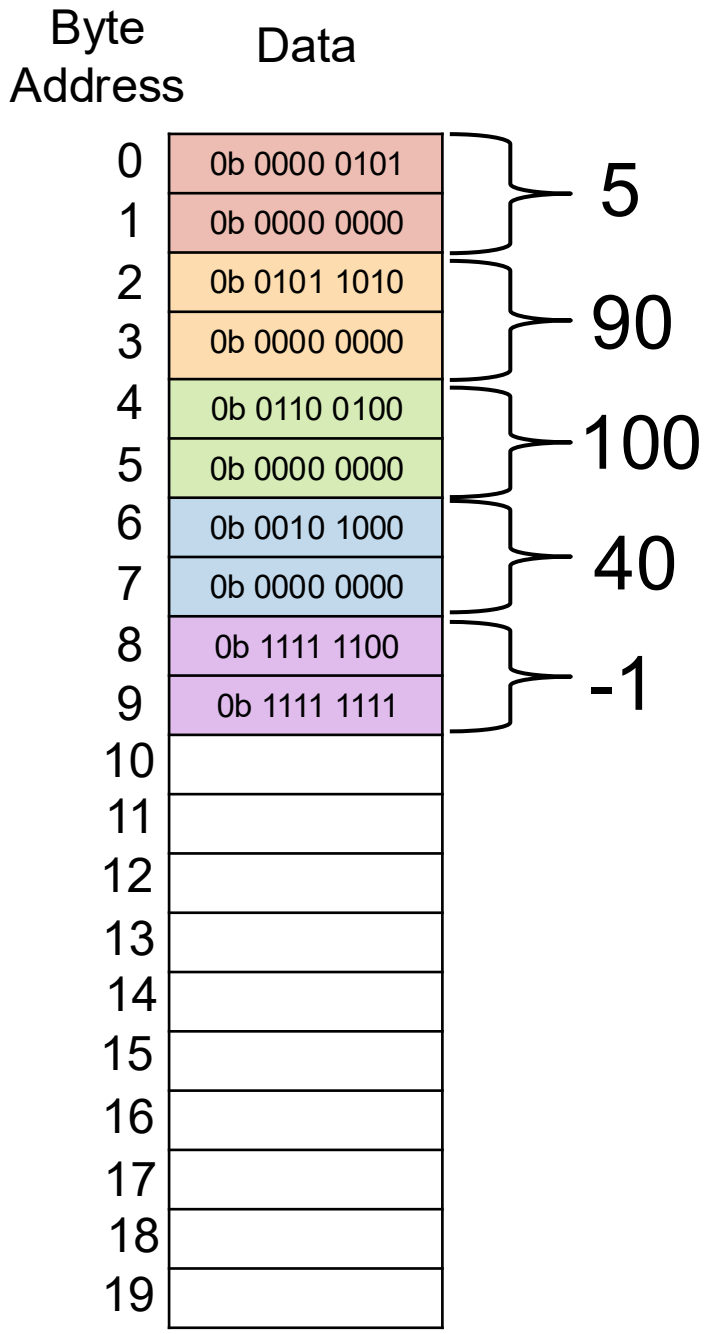
- Q: What is the value of the **half-word** (64 bits) stored at address 4?
  - 90



Arrays

0	1	2	3	4
5	90	100	40	-4

Example: Array of `int16_t`



Arrays

0	1	2	3	4
5	90	100	40	-4

Example: Array of `int8_t`

Byte Address	Data	
0	0b 0000 0101	5
1	0b 0101 1010	90
2	0b 0110 0100	100
3	0b 0010 1000	40
4	0b 1111 1100	-4
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		

Working with memory

- How do you know if an individual row should be interpreted as a one-byte value or if it is part of a larger number?

Working with memory

- How do you know if an individual row should be interpreted as a one-byte value or if it is part of a larger number?
  - **You cannot tell just by looking at memory.**
  - Main memory is simply a sequence of bytes. It does not inherently contain information about data types or boundaries between values.
  - The meaning of a particular memory region (for example, whether those bytes represent a character, an integer, or part of an array) comes from how the program and the processor interpret it. That information is stored elsewhere (in variable declarations, data type definitions, or other program metadata).
  - **When the CPU or a program accesses memory, it always specifies both:**
    - **the starting address, and**
    - **the size of the data to read or write.**

Array Indexing and Pointer Arithmetic

Array Indexing and Pointer Arithmetic

- In C, **array indexing** and **pointer arithmetic** are two views of the same thing.
- If p is a pointer an an array:

```
int *p;   // pointer to int
```

- Then

`p[i]` is equivalent to `*(p + i)`

- Both expressions mean “the element i positions after what p points to.”

Array Indexing and Pointer Arithmetic

```
int arr[] = {10, 20, 30, 40, 50};
int *p = arr;
```

Expression	Meaning	Value
`p`	address of `arr[0]`	points to 10
`p + 2`	address of `arr[2]`	points to 30
`*(p + 2)`	value at that address	30
`p[2]`	shorthand for `*(p + 2)`	30

Statically Allocated 2D Array

Statically Allocated 2D Arrays

- Here, matrix is a 2D array of int values with 50 rows and 100 columns

```
int matrix[50][100];
```

- To access an individual element, indicate both the row and the column index:

```
val = matrix[3][7]; // get int value in row 3, column 7 of matrix
```

Statically Allocated 2D Arrays

- Programs often access the elements of a 2D array by iterating with nested loops. For example, the following nested loop initializes all elements in matrix to 0:

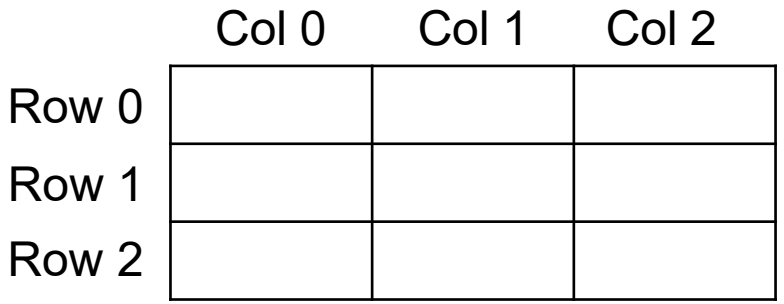
```
#define NUM_ROWS 50
#define NUM_COLS 100

int main() {
int32_t matrix[NUM_ROWS][NUM_COLS];
    int32_t i, j;
    for (i = 0; i < NUM_ROWS; i++) {
        // for each row i
        for (j = 0; j < NUM_COLS; j++) {
            // iterate over each column element in row i
            matrix[i][j] = 0;
        }
    }
    return 0;
}
```

2D Array Example

```
#define NUM_ROWS 3
#define NUM_COLS 3

int main() {
        int32_t matrix[NUM_ROWS][NUM_COLS];
        int32_t i, j;
        for (i = 0; i < NUM_ROWS; i++) {
                for (j = 0; j < NUM_COLS; j++) {
                        matrix[i][j] = i;
                }
        }
        return 0;
}
```



2D Array Example

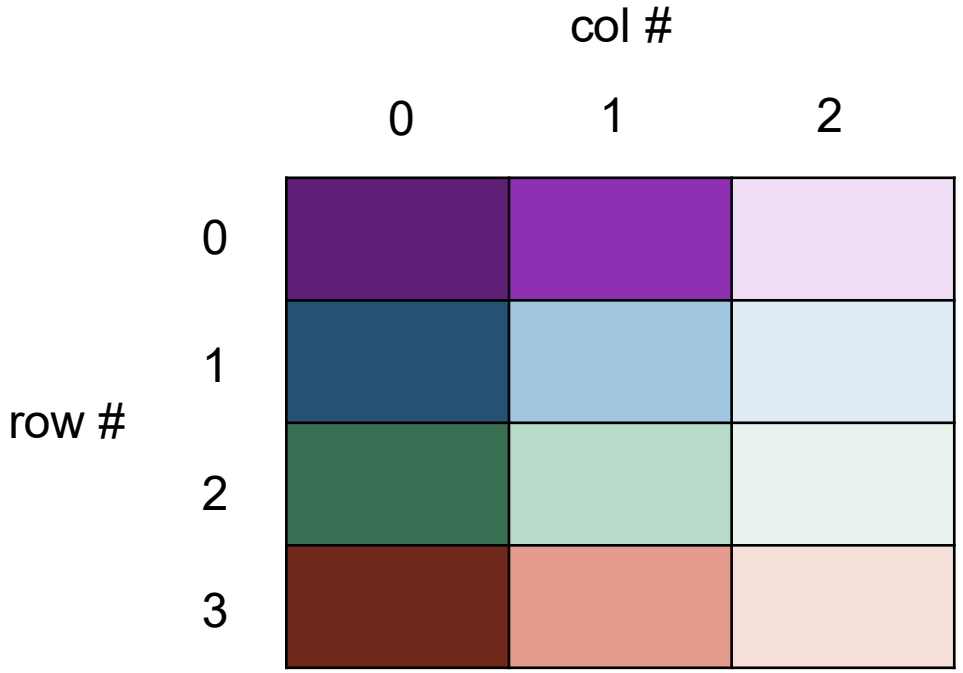
```
#define NUM_ROWS 3
#define NUM_COLS 3

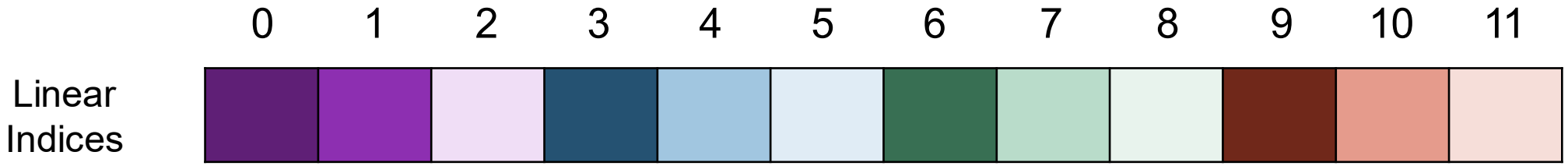
int main() {
        int32_t matrix[NUM_ROWS][NUM_COLS];
        int32_t i, j;
        for (i = 0; i < NUM_ROWS; i++) {
                for (j = 0; j < NUM_COLS; j++) {
                        matrix[i][j] = i;
                }
        }
        return 0;
}
```

	Col 0	Col 1	Col 2
Row 0	0	0	0
Row 1	1	1	1
Row 2	2	2	2

2D Array Stored in Row-Major Order

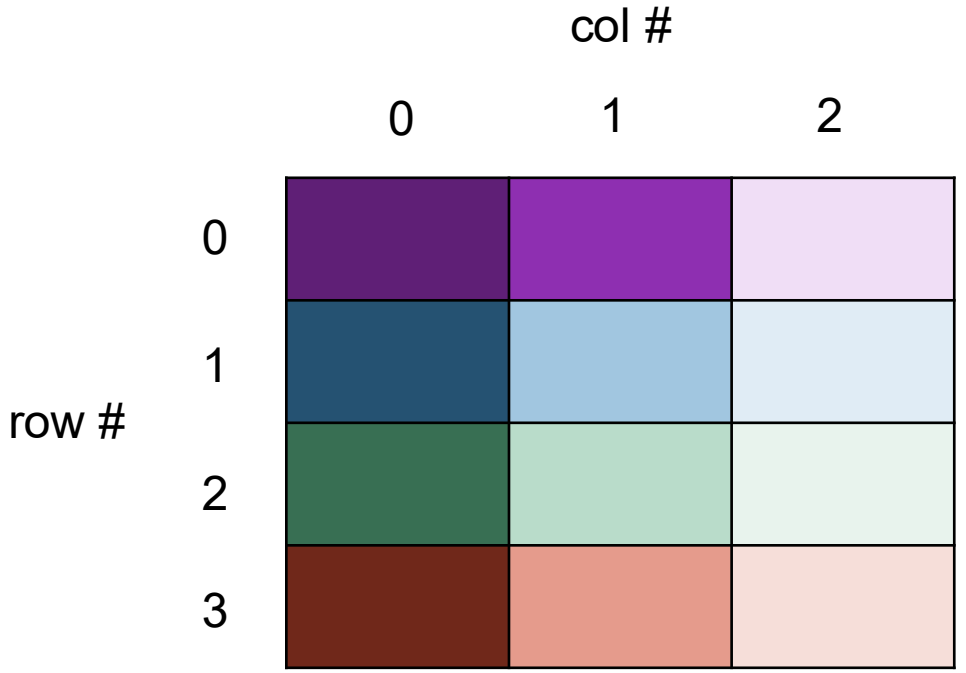
Row-major order is the method of storing multi-dimensional arrays in contiguous memory such that all elements of a row are stored together, one after another, before moving to the next row.

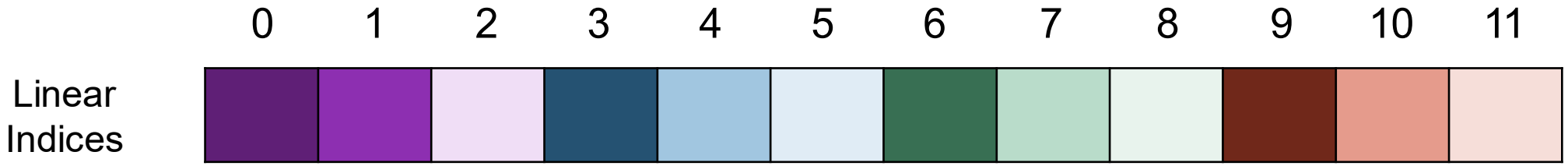




2D Array Stored in Row-Major Order

- The index of element [0][1] is 1
- The index of element [1][1] is 4
- The index of element [2][2] is 8





Computing the Linear Index

- Q1. What is the formula to compute the **linear index** of an element in a 2D array stored in row-major order?
  - R = number of rows
  - C = number of columns
  - Indices: [i][j]
  - Linear index =
- Q2. Given a 2D array of size 20 × 30, what is the linear index of the element at position [15][20]?
  - Linear index =

Computing the Address

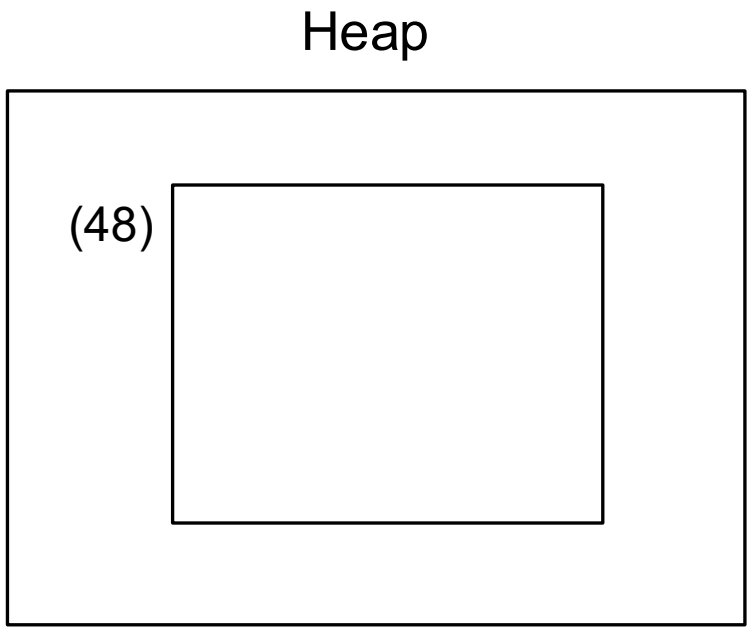
- Q1. What is the formula to compute the address of an element in a 2D array stored in row-major order? Parameters:
  - R = number of rows
  - C = number of columns
  - Indices: [i][j]
  - Base Address
  - Element size
  - Address of any element =
- Q2. If you have a 2D array declared as `int32_t A[100][5]` and the base address of the array (A[0][0]) is 0xFFFF00000000, what is the address of the element A[3][2]?

Dynamically Allocated 2D Arrays

Dynamically Allocated 2D Array (Method 1)

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12

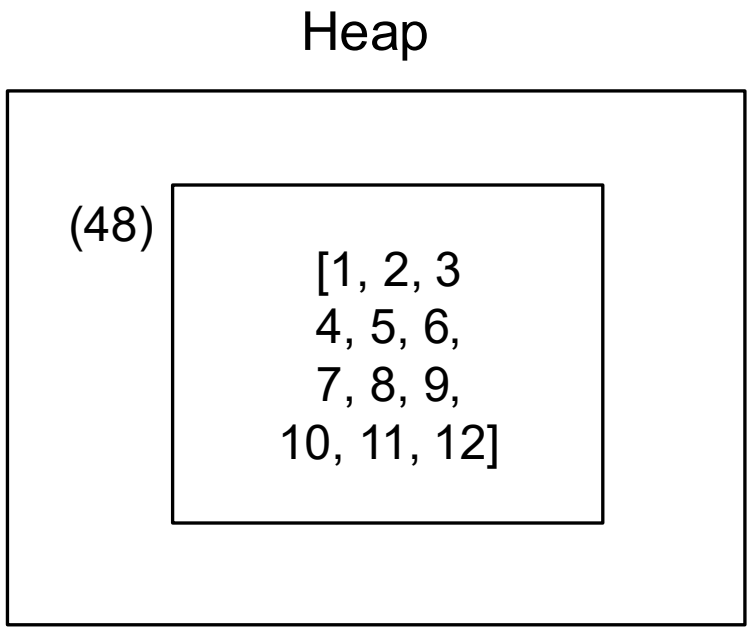
```
sizeof(int) = 4 bytes
```



Dynamically Allocated 2D Array (Method 1)

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12

```
sizeof(int) = 4 bytes
```



Dynamically Allocated 2D Array (Method 1)

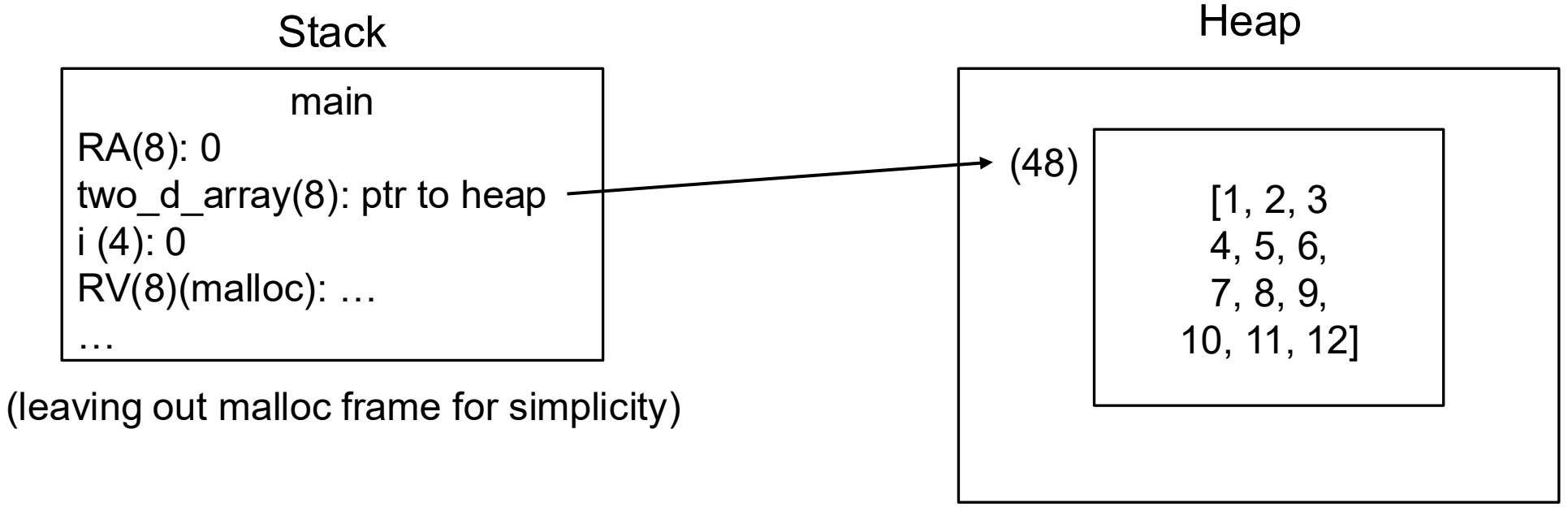
```
#define NUM_ROWS 3
#define NUM_COLS 3

int main(void) {
    int* two_d_array;

    two_d_array = malloc(sizeof(int) * NUM_ROWS * NUM_COLS);
}
```

the type is a pointer to an int (the element type)

allocate in a single malloc of N x M int-sized elements



(leaving out malloc frame for simplicity)

```
git clone
https://github.com/COMP211-
FA26/lecture.git
```

method1.c

1. The grid is allocated as one contiguous block of memory. How many ints are allocated for a grid with rows = 4 and cols = 6?
2. Why do we need the function idx in this program? Why can't we write grid[i][j]?
3. For a grid with 6 columns, what does idx(2, 3, 6) return?
4. Suppose grid points to address 1000 and an int occupies 4 bytes. What address would contain grid[2][3] for a grid with 6 columns? Assume sizeof(int) = 4.
5. Describe what in_bounds checks. Give one example for which it returns true and one for which it returns false.
6. Look at main. How will the provided code help you determine whether your implementations of init_zero, grid_set, and grid_get are working correctly?

Static Inline Functions

- A **static inline function** is a function the compiler is encouraged to:
  - Inline: replace the function call with the actual code (to avoid call overhead)
  - Keep local to the file where it is defined (because of static)

Keyword	Meaning	Why we want it
inline	Suggests expanding the function at the call site	Faster execution (no function-call overhead)
static	Limits visibility to this source file	Avoids linker conflicts in headers

- When to use
  - Small, frequently used helper functions, especially in performance-sensitive code.

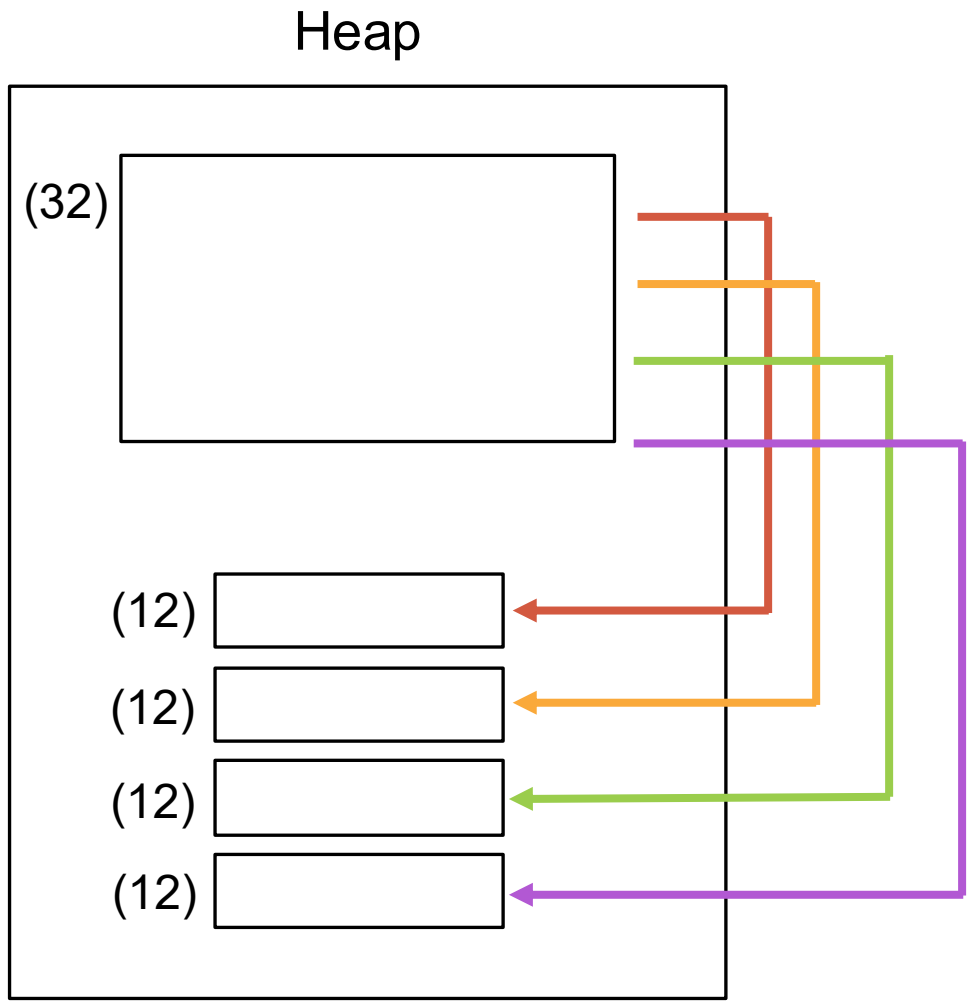
method1.c

- Complete init_zero so that every element of the grid is initialized to 0.
- Complete grid_set so that it:
  - checks whether (i, j) is a valid location,
  - stores value at that location if it is valid, and
  - calls error_oob if it is not valid.
- Complete grid_get so that it:
  - checks whether (i, j) is a valid location,
  - returns the value at that location if it is valid, and
  - calls error_oob if it is not valid.

Dynamically Allocated 2D Array (Method 2)

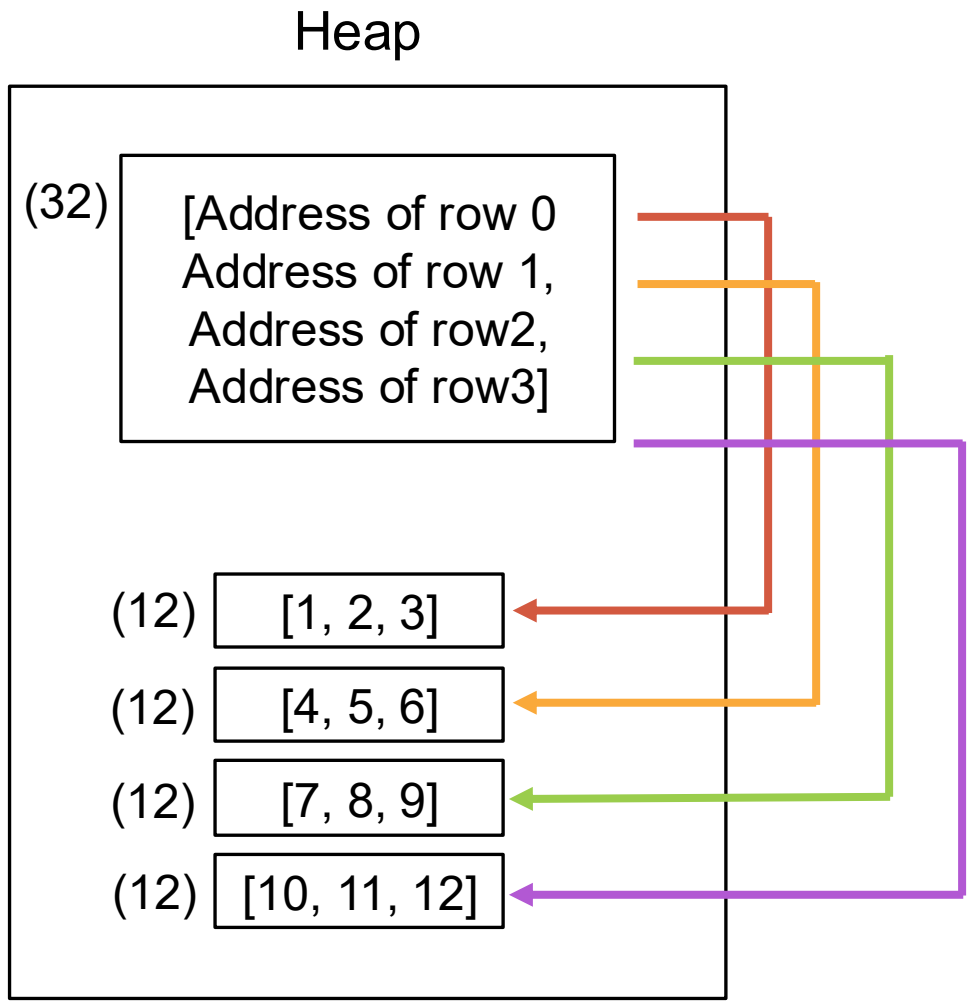
	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12

```
sizeof(int) = 4 bytes
```



Dynamically Allocated 2D Array (Method 2)

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9
Row 3	10	11	12



Dynamically Allocated 2D Array (Method 2)

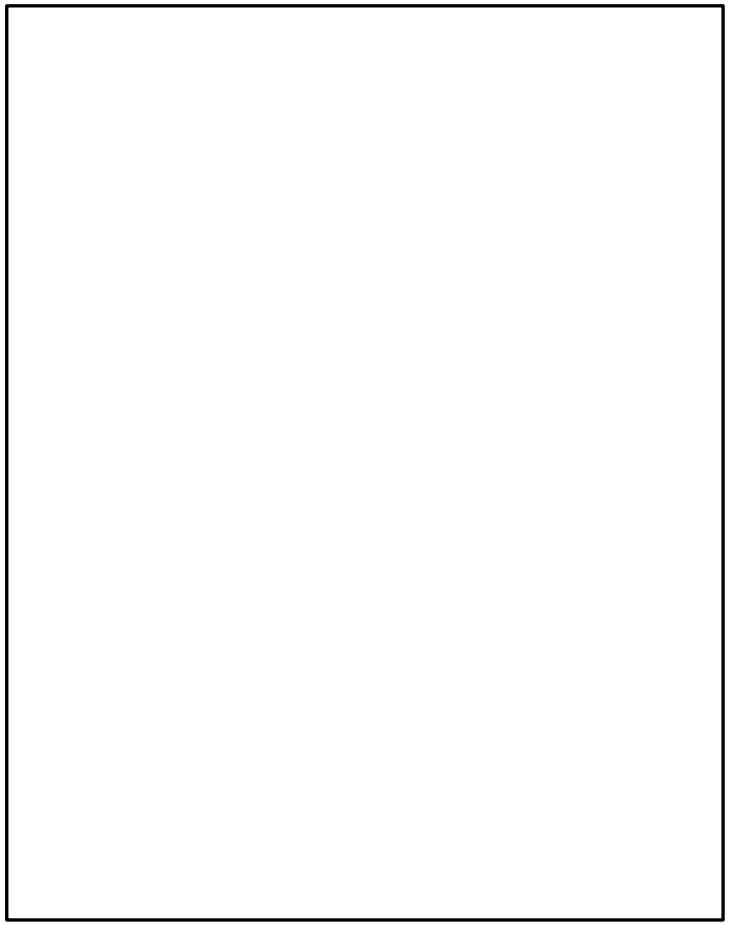
```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

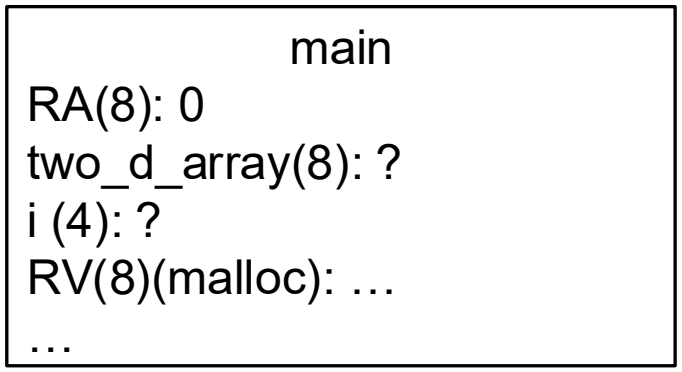
    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```

Heap



Stack



(leaving out malloc frames for simplicity)

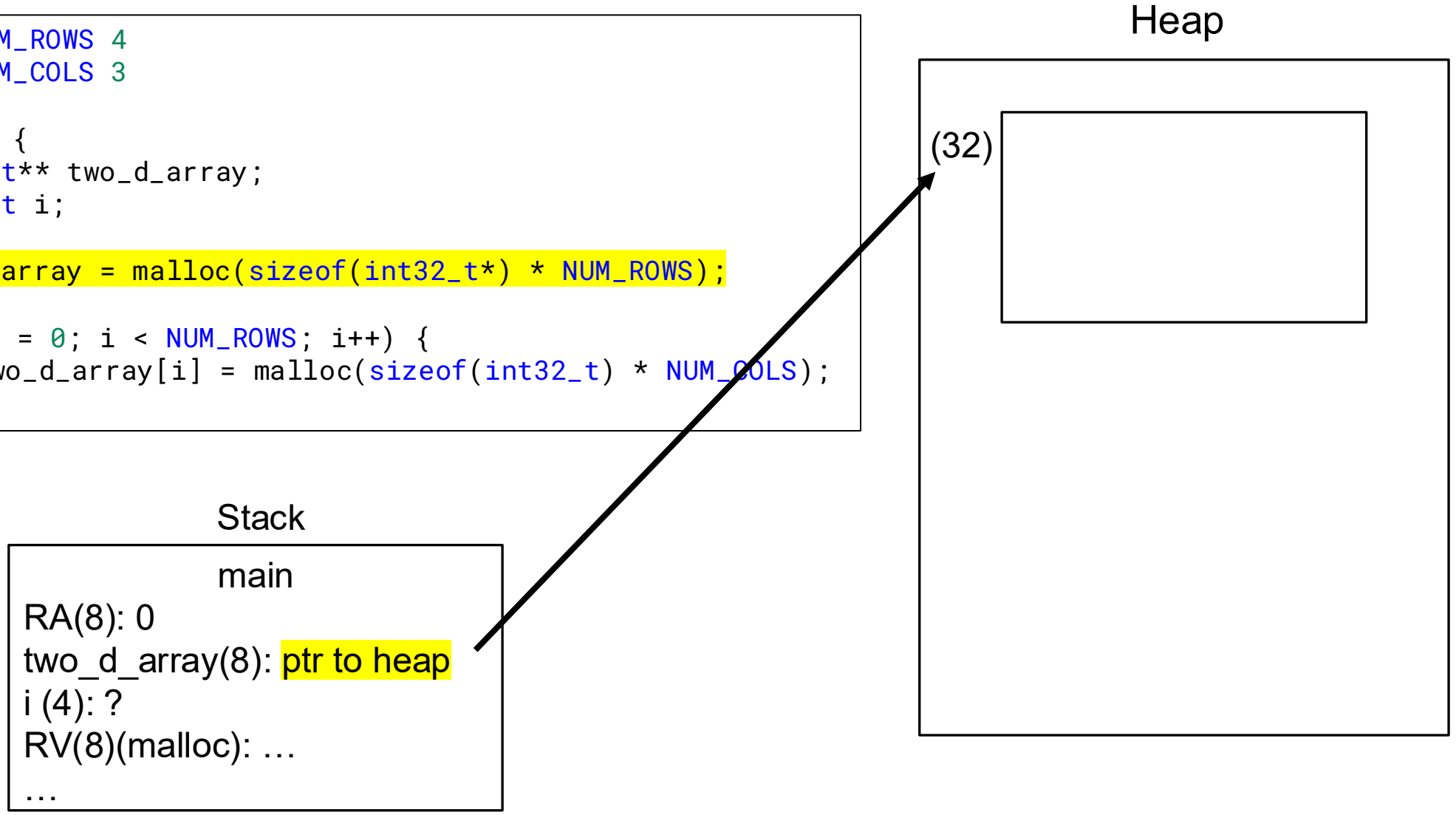
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

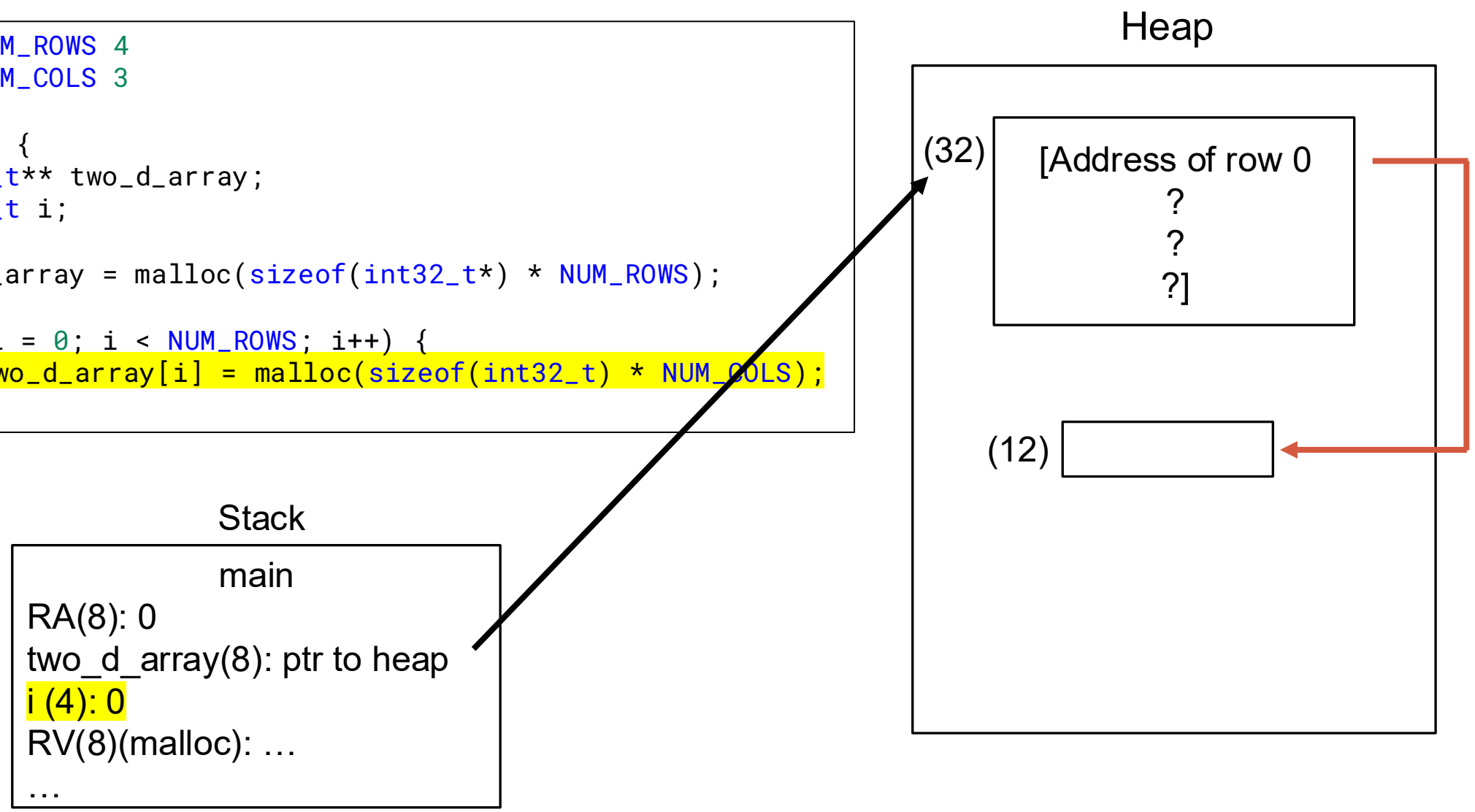
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

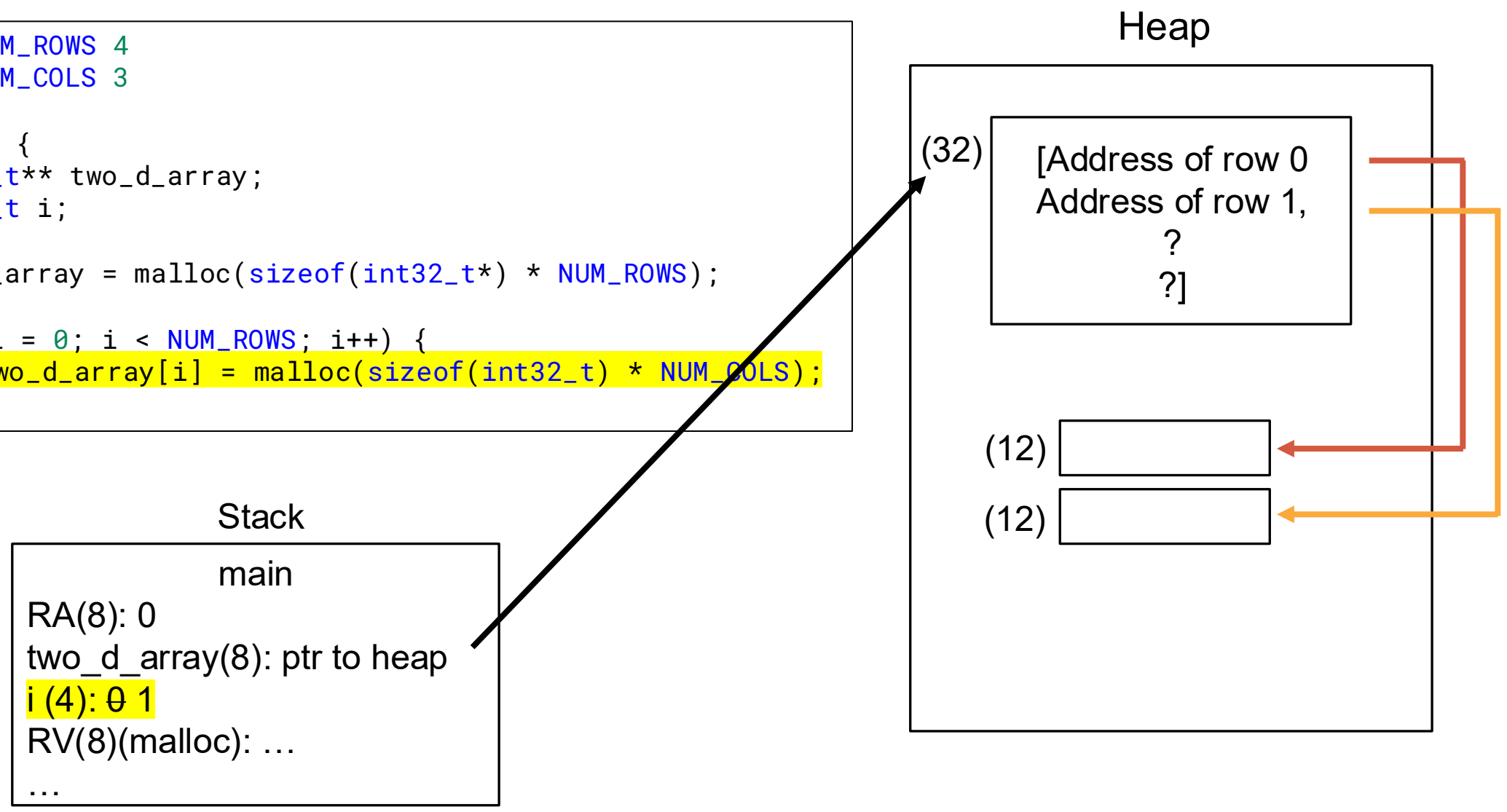
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

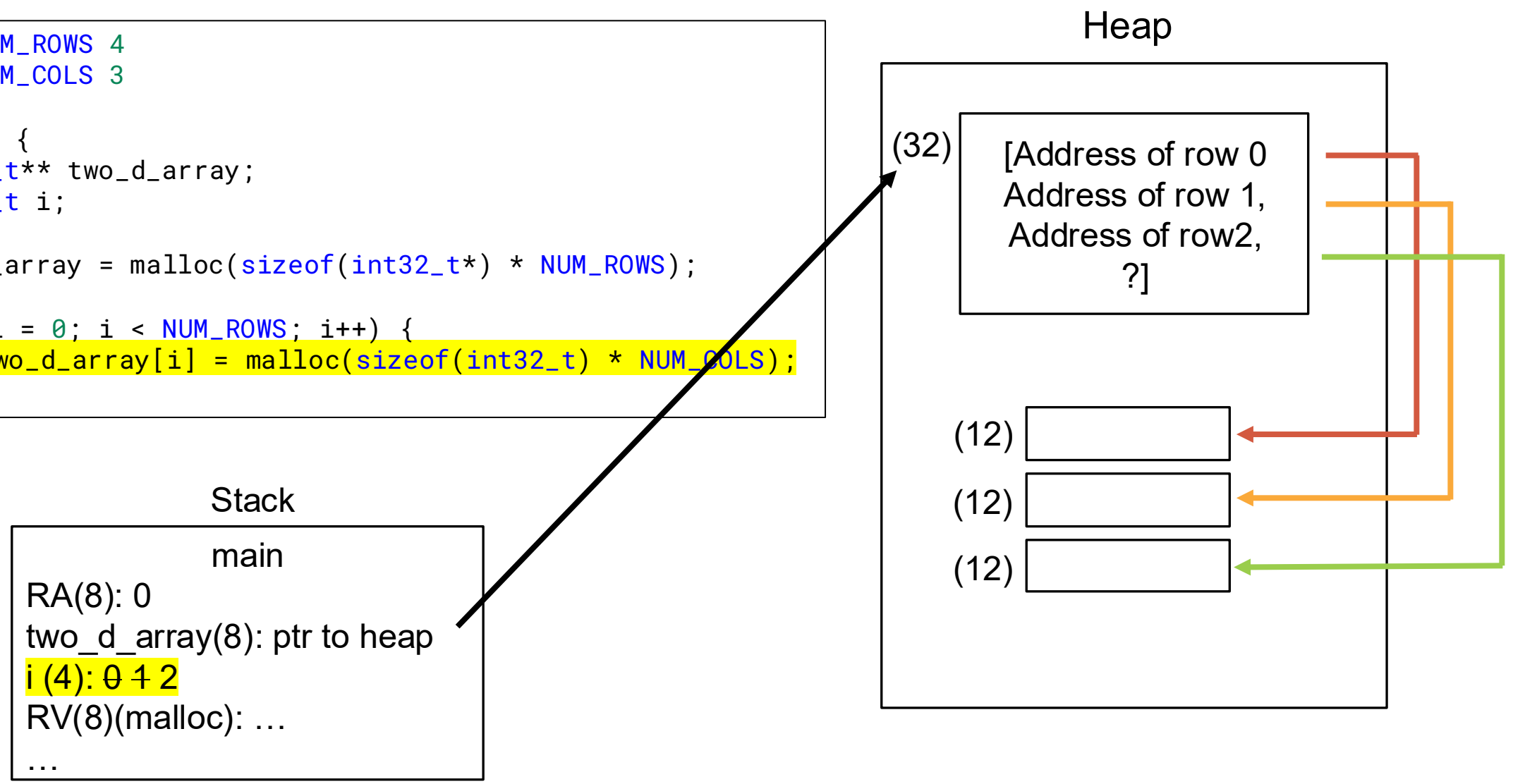
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

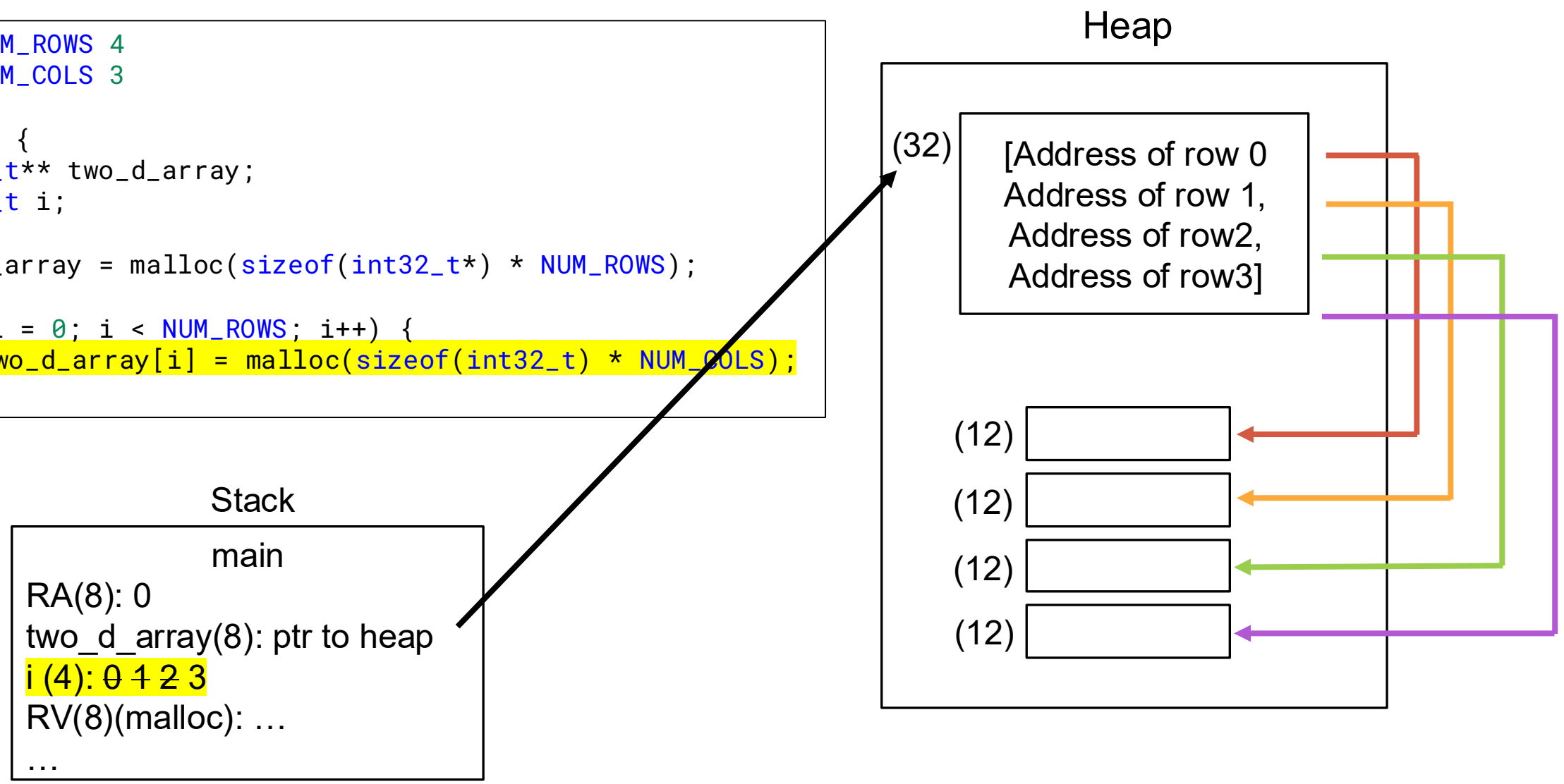
Dynamically Allocated 2D Array (Method 2)

```c
#define NUM_ROWS 4
#define NUM_COLS 3

int main() {
    int32_t** two_d_array;
    int32_t i;

    two_d_array = malloc(sizeof(int32_t*) * NUM_ROWS);

    for (i = 0; i < NUM_ROWS; i++) {
        two_d_array[i] = malloc(sizeof(int32_t) * NUM_COLS);
}
```



(leaving out malloc frames for simplicity)

method2.c

- Implement free2d.
- Implement swap_rows.

Swap example

```
Before swap_rows(a, 5, 1, 4):
   0   1   2   3   4   5
  10  11  12  13  14  15
  20  21  22  23  24  25
  30  31  32  33  34  35
  40  41  42  43  44  45

After swap:
   0   1   2   3   4   5
  40  41  42  43  44  45
  20  21  22  23  24  25
  30  31  32  33  34  35
  10  11  12  13  14  15
```